Butterfly vs Dense Layers on MNIST

A neural network library from scratch in C++

Valtteri Haikarainen

Algorithms and AI lab course
University of Helsinki

Demo session, 2026

The question

Goal

Can a **butterfly-structured** linear layer replace a **dense** linear layer in a small neural network without a meaningful loss in accuracy, while using far fewer parameters and operations?

- Everything implemented by hand in C++17: tensors, forward and backward passes, loss, optimizer
- No machine-learning libraries, no automatic differentiation
- **Hypothesis:** butterfly within $\sim$1 percentage point of dense on MNIST, with $\sim 27\times$ fewer hidden-layer parameters

Why dense layers are expensive

A dense layer computes

$$y = xW + b, \qquad W \in \mathbb{R}^{n \times n}$$

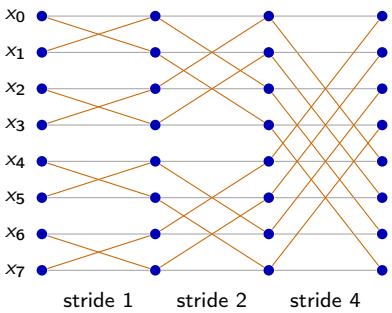
- every output depends on every input
- n^2 weights, n^2 multiply-adds per input
- doubling the width $\Rightarrow 4\times$ the cost

Idea from the FFT: the discrete Fourier transform is also a full $n \times n$ matrix, yet it can be applied in $O(n \log n)$:

- split the work into $\log_2 n$ *stages*
- each stage only mixes *pairs* of positions

Butterfly layer = FFT wiring, but the 2×2 mixing blocks are **learned**.

The butterfly layer



$n = 8$: three stages

In stage s, position i is paired with $j = i \oplus 2^s$:

$$\begin{pmatrix} y_i \\ y_j \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_i \\ x_j \end{pmatrix}$$

- $\log_2 n$ stages $\times$ $n/2$ blocks $\times$ 4 weights
- after all stages every output depends on every input

n	dense n^2	butterfly $2n \log_2 n$
64	4,096	768
512	262,144	9,216
4096	16,777,216	98,304

Implementation

Components

- `Tensor`: row-major float array, `matmul`, `sum`
- `DenseLinear`, `ButterflyLinear`
- ReLU, stable softmax + cross-entropy
- Adam optimizer, He initialization
- MNIST IDX file loader
- `Mlp` and the `train_mnist` program

Each layer has `forward` (caches its inputs) and `backward` (returns gradients).

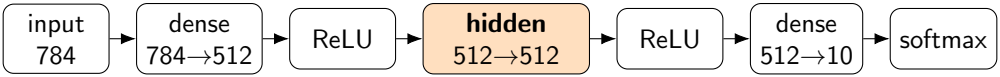
Manual backpropagation, e.g. dense layer with $g = \partial L/\partial y$:

$$\frac{\partial L}{\partial x} = gW^\top, \quad \frac{\partial L}{\partial W} = x^\top g, \quad \frac{\partial L}{\partial b} = \sum_{\text{batch}} g$$

Butterfly backward: visit the stages *in reverse* and apply the transposed 2×2 block to each pair:

$$\begin{pmatrix} \partial x_i \\ \partial x_j \end{pmatrix} = \begin{pmatrix} a & c \\ b & d \end{pmatrix} \begin{pmatrix} g_i \\ g_j \end{pmatrix}$$

Experiment



Three variants of the hidden layer:

- `none`: skipped (baseline)
- `dense`: 262,656 parameters
- `butterfly`: 9,728 parameters

Identical settings otherwise:

- 60,000 training / 10,000 test images
- 5 epochs, batch 64, Adam, lr 10^{-3}
- 3 random seeds per variant

Demo

Build and run the tests

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build
./build/tests                          # 56 tests
```

Train one variant

```
scripts/download_mnist.sh
./build/train_mnist --variant butterfly --epochs 1
./build/train_mnist --variant dense     --epochs 1
```

Output

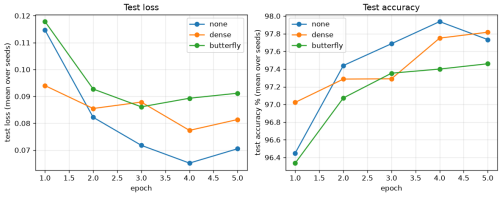
```
variant=butterfly seed=1 hidden_params=9728 total_params=416778
epoch=1 train_loss=... test_loss=... test_accuracy=... epoch_seconds=...
```

Options: `-variant`, `-epochs`, `-batch`, `-lr`, `-seed`, `-data`, `-log` (CSV)

Results

Test accuracy after 5 epochs, mean ± std over 3 seeds

Variant	Hidden params	Accuracy	s/epoch
`none`	0	97.73 ± 0.40	9.7
`dense`	262,656	97.82 ± 0.24	16.0
`butterfly`	9,728	97.46 ± 0.38	11.8



- butterfly is **0.36 pp** behind dense with **27**× fewer hidden parameters ⇒ hypothesis holds
- but `none` is just as good: MNIST is too easy to show whether the hidden layer is needed at all

Theory vs measured speed

Hidden layer	MACs / image	extra s/epoch
`dense`	262,144	+6.3
`butterfly`	9,216	+2.1

28× fewer operations, but only ∼3× faster.

- dense: ∼7.5 GMAC/s, contiguous memory, vectorized by the compiler
- butterfly: ∼0.8 GMAC/s, stride changes every stage, one tensor copy per stage
- $O(n \log n)$ wins asymptotically, but at $n = 512$ constant factors eat most of the advantage

Testing

- **Gradient checks**: every backward pass compared to central differences $\frac{f(\theta+\varepsilon)-f(\theta-\varepsilon)}{2\varepsilon}$
- **Known answer**: butterfly with blocks $\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)$ must equal the Walsh–Hadamard matrix $H_{ij} = (-1)^{\mathrm{popcount}(i \,\&\, j)}$ ($n = 8$ and 512)
- **Mutation check**: a planted transpose bug was caught by the gradient tests
- **Integration**: the network must learn synthetic Gaussian clusters

Numbers

- 56 GoogleTest tests, all passing
- 94 % line coverage (gcov / gcovr)

Also tested

softmax stability at ± 1000, hand-computed losses, Adam per-parameter state, seed reproducibility, MNIST loader errors

Bugs that taught me something

1. **No learning at all** (loss stuck at 2.30): butterfly weights initialized with std 0.02 shrink the signal $\sim 35\times$ per stage, over 9 stages. Fix: std $1/\sqrt{2}$ keeps the variance constant.
2. **Degenerate comparison**: without a ReLU after the hidden layer, hidden and output layer collapse into one linear map.
3. **Adam**: one shared step counter broke bias correction; now each parameter has its own state.
4. **Training seemed to hang**: every element access allocated a `std::vector`. Allocation-free indexing gave a **120**$\times$ speed-up.

Conclusions

- A learnable butterfly layer works: 97.5 % on MNIST with 9,728 instead of 262,656 hidden parameters
- Fewer operations do not automatically mean faster code; memory access patterns matter
- Gradient checks were the most valuable tests

Future work

- a harder task where the hidden layer matters (e.g. Fashion-MNIST)
- faster butterfly kernel: in-place stages, no bounds checks
- larger widths, where $n \log n$ vs n^2 really pays off

Questions?